

- **Optimization-based attacks:** Existing optimization-based attacks [6, 5, 24, 41] primarily use GCG (or nano-GCG) [41] to optimize a suffix. By default, we append the optimized suffix to the end of the injected instruction. We run nano-GCG to optimize the suffix until the attack succeeds or a maximum of 500 iterations is reached. Implementation details are in Appendix E.

Unless otherwise mentioned, we use Combined Attack as it can be applied to closed-source LLMs. We insert an injected prompt into a context at a random location, as this is the most general case.

Backend LLMs: We evaluate 3 open-source LLMs: Llama-3.1-8B-Instruct, Llama-3.1-70B-Instruct, and Qwen3-Omni-30B-A3B-Instruct. We also evaluate 4 closed-source LLMs: GPT-4o, GPT-4o-mini, GPT-4.1, and GPT-5. GPT-4o-mini is trained to improve the LLM’s ability to resist prompt injection by OpenAI [62, 16]. Unless otherwise mentioned, we use Llama-3.1-8B-Instruct as the backend LLM.

Baselines: We compare with state-of-the-art baselines.

- **Prevention-based defenses:** We compare with Sandwich prevention [11], Instructional prevention [12], and Meta-SecAlign [17]. We exclude StruQ [14] and SecAlign [43], since Meta-SecAlign is an improved defense upon these two baselines and outperforms them. We also compare with DataFilter [50] (a concurrent work).
- **Detection-based defenses:** We evaluate DataSentinel [32] (state-of-the-art detection method), PromptGuard [29] (released by Meta), and AttentionTracker [31].
- **Detection-based + attribution-based defenses:** We compare with PromptArmor [36] and PromptLocate [37], which already integrate detection-based defenses in their design (e.g., PromptLocate leverages DataSentinel). Wang et al. [35] proposed AttnTrace, which can trace back to malicious texts leading to the output of an LLM. We combine AttnTrace with state-of-the-art detection methods. Once a context is detected as contaminated, we use AttnTrace to trace back to texts responsible for an LLM output and remove these texts from the context.

We use open-source implementations for baselines. As there is no open-source implementation for PromptArmor, we implement it ourselves and use GPT-4o for prompt injection detection and removal.

Evaluation metrics: We use following evaluation metrics.

- **Utility for target tasks (Utility):** This metric measures the utility of an LLM for a target task with a clean (or contaminated) context. A defense is effective if it can (1) maintain the utility for clean contexts (i.e., without attacks), and (2) achieve a high utility under contaminated contexts, i.e., reducing the effectiveness of the injected instruction to make the LLM still successfully perform the target task under contaminated contexts. Note that the utility metric is task dependent. We adopt the corresponding metrics provided by the LongBench [54]. In particular, we use F1-Score for question answering tasks, ROUGE-L [63] for summarization tasks, and exact match accuracy for information retrieval task. We use Edit Sim (Levenshtein distance) for code generation tasks, which is widely used in code generation evaluation [64].
- **Attack success rate (ASR):** This metric measures the success of a prompt injection attack. In particular, given a target task and a contaminated context, the ASR is 1 if the backend LLM follows the injected instruction, and 0 otherwise. A defense is less effective if ASR is high under attacks. We use different ASR metrics for different injected tasks. We adopt task-specific ASR definitions following prior work [17]. For *target answer generation* and *hacked generation*, an attack is considered successful if the model output begins with the target answer or the word “Hacked”. For *classic NLP task* and *general instruction*, we use the LLM-as-a-judge to determine whether the output follows the injected instruction. The evaluation prompt and implementation details are provided in Appendix D.
- **Runtime:** This metric measures the efficiency of a defense. Given a target instruction and a context, it measures the overall computation time (s) in getting the output of a backend LLM under a defense. For PISanitizer, the runtime includes both context sanitization and the backend LLM’s output generation for a target task based on the sanitized context. The runtime is calculated on a 96GB H100 GPU.

We report the average value for the above metrics. Note that for detection-based baselines, we report false positive rate (FPR) and false negative rate (FNR). FPR (or FNR) measures the fraction of clean (or contaminated) contexts that are falsely detected as contaminated (or clean).

Hyper-parameter settings: PISanitizer mainly involves the design of sanitization instruction and an LLM for sanitization. Unless otherwise specified, we use the sanitization instruction introduced in Section 4.1 and use Llama-3.1-8B-Instruct to sanitize injected tokens in a context. Our complete algorithm also includes additional hyperparameters, such as the smooth window sizes w_s , threshold θ , and peak distance d . Details are provided in Appendix B. We repeatedly apply PISanitizer to a context at most 5 times.

Table 1: PISanitizer is effective across multiple datasets and prompt injection attacks.

Dataset	Attack	No Defense		PISanitizer	
		Utility	ASR	Utility	ASR
Qasper	No Attack	0.32	0.0	0.32	0.0
	Naive Attack	0.18	0.82	0.31	0.0
	Escape Character	0.18	0.89	0.29	0.0
	Context Ignoring	0.14	0.92	0.29	0.0
	Fake Completion	0.16	0.89	0.30	0.0
	Combined Attack	0.15	0.92	0.31	0.0
	GCG Attack	0.12	0.96	0.28	0.02
HotpotQA	No Attack	0.59	0.0	0.59	0.0
	Naive Attack	0.30	0.63	0.56	0.04
	Escape Character	0.28	0.62	0.58	0.0
	Context Ignoring	0.24	0.72	0.59	0.0
	Fake Completion	0.32	0.57	0.59	0.02
	Combined Attack	0.24	0.66	0.59	0.01
	GCG Attack	0.12	0.96	0.56	0.02
GovReport	No Attack	0.34	0.0	0.34	0.0
	Naive Attack	0.03	0.98	0.35	0.0
	Escape Character	0.02	1.0	0.34	0.0
	Context Ignoring	0.02	0.97	0.34	0.0
	Fake Completion	0.07	0.85	0.34	0.0
	Combined Attack	0.03	0.97	0.34	0.0
	GCG Attack	0.02	1.0	0.34	0.0
MultiNews	No Attack	0.28	0.0	0.28	0.0
	Naive Attack	0.07	0.82	0.28	0.01
	Escape Character	0.05	0.91	0.28	0.0
	Context Ignoring	0.12	0.63	0.28	0.0
	Fake Completion	0.09	0.77	0.28	0.0
	Combined Attack	0.05	0.91	0.28	0.0
	GCG Attack	0.03	1.0	0.28	0.0
LCC	No Attack	0.42	0.0	0.42	0.0
	Naive Attack	0.31	0.33	0.37	0.04
	Escape Character	0.28	0.42	0.35	0.03
	Context Ignoring	0.35	0.35	0.40	0.01
	Fake Completion	0.21	0.62	0.37	0.01
	Combined Attack	0.15	0.73	0.37	0.0
	GCG Attack	0.10	0.82	0.38	0.0
Passage Retrieval	No Attack	1.0	0.0	1.0	0.0
	Naive Attack	0.68	0.31	0.96	0.01
	Escape Character	0.68	0.32	0.97	0.01
	Context Ignoring	0.61	0.37	0.96	0.01
	Fake Completion	0.60	0.36	0.96	0.02
	Combined Attack	0.67	0.27	0.98	0.01
	GCG Attack	0.05	0.94	0.97	0.0

5.2 Main Results

PISanitizer is effective and maintains utility: As shown in Table 1, PISanitizer demonstrates strong effectiveness across different datasets and prompt injection attacks. When there are no attacks, the utility of PISanitizer is similar to that without defense, demonstrating that PISanitizer maintains utility. Under prompt injection attacks, PISanitizer can reduce the ASR to nearly zero. Moreover, the utility of PISanitizer for contaminated contexts is similar to that without attacks. This means that PISanitizer successfully

Table 2: PISanitizer is effective for different LLMs under Combined Attack. We omit “-Instruct” from the names of LLMs for space reasons. Utility represents the Utility for target tasks under no attacks and defenses.

Dataset	LLM	No Defense		PISanitizer		Utility
		Utility	ASR	Utility	ASR	
Qasper	Llama-3.1-8B	0.15	0.92	0.31	0.0	0.32
	Llama-3.1-70B	0.35	0.54	0.39	0.0	0.40
	Qwen3-Omni-30B	0.24	0.71	0.27	0.01	0.27
	GPT-4o	0.38	0.10	0.40	0.0	0.40
	GPT-4o-mini	0.36	0.26	0.30	0.0	0.31
	GPT-4.1	0.42	0.21	0.40	0.0	0.40
	GPT-5	0.46	0.13	0.45	0.0	0.45
HotpotQA	Llama-3.1-8B	0.24	0.66	0.59	0.01	0.59
	Llama-3.1-70B	0.55	0.17	0.67	0.02	0.68
	Qwen3-Omni-30B	0.52	0.29	0.61	0.01	0.62
	GPT-4o	0.75	0.04	0.71	0.01	0.73
	GPT-4o-mini	0.67	0.17	0.72	0.01	0.73
	GPT-4.1	0.72	0.06	0.69	0.01	0.69
	GPT-5	0.84	0.01	0.81	0.0	0.81
GovReport	Llama-3.1-8B	0.03	0.97	0.34	0.0	0.34
	Llama-3.1-70B	0.03	0.96	0.26	0.0	0.26
	Qwen3-Omni-30B	0.08	0.71	0.22	0.0	0.22
	GPT-4o	0.23	0.22	0.31	0.0	0.31
	GPT-4o-mini	0.30	0.0	0.30	0.0	0.30
	GPT-4.1	0.26	0.17	0.31	0.0	0.31
	GPT-5	0.32	0.0	0.32	0.0	0.32
MultiNews	Llama-3.1-8B	0.05	0.91	0.28	0.0	0.28
	Llama-3.1-70B	0.06	0.79	0.21	0.0	0.21
	Qwen3-Omni-30B	0.03	1.0	0.18	0.0	0.18
	GPT-4o	0.21	0.11	0.23	0.0	0.23
	GPT-4o-mini	0.23	0.0	0.23	0.0	0.23
	GPT-4.1	0.22	0.03	0.22	0.0	0.22
	GPT-5	0.23	0.01	0.23	0.0	0.23
LCC	Llama-3.1-8B	0.15	0.73	0.37	0.0	0.42
	Llama-3.1-70B	0.22	0.63	0.42	0.02	0.42
	Qwen3-Omni-30B	0.08	0.85	0.52	0.0	0.52
	GPT-4o	0.38	0.50	0.74	0.0	0.76
	GPT-4o-mini	0.39	0.50	0.72	0.0	0.72
	GPT-4.1	0.20	0.73	0.72	0.0	0.75
	GPT-5	0.24	0.68	0.73	0.0	0.75
Passage Retrieval	Llama-3.1-8B	0.67	0.27	0.98	0.01	1.0
	Llama-3.1-70B	0.97	0.01	0.98	0.0	0.98
	Qwen3-Omni-30B	1.0	0.0	1.0	0.0	1.0
	GPT-4o	0.99	0.0	0.99	0.0	0.99
	GPT-4o-mini	0.99	0.0	0.98	0.0	0.99
	GPT-4.1	0.98	0.01	0.99	0.0	0.99
	GPT-5	0.99	0.0	0.99	0.0	0.99